

Wind Estimation by Multirotor Drone State using Machine Learning with Data Rotation and Reduction

Steven Zimmerman^{*a}, Ryoza Nagamune^a, and Steven Rogak^a

^aDepartment of Mechanical Engineering, University of British Columbia
2054-6250 Applied Science Lane, Vancouver, BC, Canada, V6T 1Z4

^{*}Corresponding Author, steven.zimmerman@ubc.ca

ACKNOWLEDGEMENTS

This research is made possible by funding from the John Tiedje fellowship administered by the Clean Energy Research Center (CERC) at the University of British Columbia, and by funding from the FlareNET strategic network funded by the Natural Sciences and Engineering Research Council of Canada (NSERC).

Wind Estimation by Multirotor Drone State using Machine Learning with Data Rotation and Reduction

Anonymized Manuscript

Abstract—The problem of wind estimation by multirotor drone is suitable for machine learning, because of the unknown drag coefficient which changes with orientation. Here, we present the application of data rotation and reduction to model training, in order to improve the performance of models in generalized flight conditions. Rotation allows the model to learn rotational invariance to arbitrary global coordinates, while reduction addresses data imbalance and decreases training time. Two models are trained on experimental flight data: a gated-recurrent-unit (GRU) and a long-short-term-memory (LSTM) neural-network. The better performing GRU achieved 0.48 m/s root-mean-square-error on new flight data, although the LSTM performs similarly. These models approach the experimental limits of performance, which is determined by the spatial variation of wind as measured by the variation in two separate anemometer readings.

Keywords: Wind Estimation, Multirotor Drone, Machine-Learning, Long-Short-Term-Memory, Neural-Network, Data Augmentation

I. INTRODUCTION

Measuring a locally varying wind field is important in a number of fields and applications, such as environmental monitoring, wind farming, control systems engineering, and others. In environmental studies, wind measurement can be coupled with gas concentration to produce flux emissions measurements. This can be used to quantify the emissions rate of known sources or to find unknown sources. Many papers [1, 2, 3] have presented results on this, typically relying on a mobile robotics platform to sample the concentration and measuring the wind by a static anemometer with a spatial wind uniformity assumption. A review is provided by Shaw [4]. Knowledge of the changing wind field at the location of gas measurement provides significant benefits to these fields. In wind farming applications, wind measurement can be used to inform wind turbine placement, design, and control [5, 6]. This is particularly important as the wind field around complex terrain or buildings is not easily modelled. In control systems engineering, flying robotics platforms that are susceptible to wind can benefit from real-time estimation of the disturbance wind field [7, 8]. Control algorithms can be created to modify a desired trajectory in order to take advantage of the wind field for higher flight efficiency, or to reject this disturbance. Each of these applications requires a method of experimentally measuring the wind field on a local scale, with high positional accuracy.

Some common methods of measuring the wind field include small, fixed wing aircraft or static anemometers on

large booms, but both of these have their own trade-offs. These methods represent two ends of a spectrum, where the static anemometer measures detailed information about one location and a fixed wing aircraft measures coarse information about a larger number of locations due to their fast movement speed. Multirotor drones offer an intermediate trade-off in this spectrum, as they have high positional versatility and control. They can be deployed at a variety of sites to conduct surveys, and typically have the payload capacity for a variety of sensors. Thus, they offer a very attractive option for wind measurement, but this suffers when attaching an anemometer as a payload. The downwash from the propellers creates local airflow over an anemometer, disturbing the measurement of the free-stream wind. It is possible to attach an anemometer on a boom away from the propellers [9], but this adds weight and flight control penalties. A method to overcome this established in literature is by implementing a disturbance observer that can estimate the wind implicitly from the multirotor state and control inputs, eliminating the need for a dedicated anemometer.

Neuman [10] presented the pioneering work on this problem, demonstrating a disturbance observer based on a simplified physics representation of a multirotor. This work used a static assumption, directly relating the orientation to the wind magnitude and direction through wind tunnel testing. However this work does not generalize well to dynamic flight phases. Allison [11] improved upon these results by applying a machine learning (ML) long-short-term-memory (LSTM) neural-network (NN) model to a simulated multirotor predicting Dryden wind based on orientation alone. Wang [12] conducted an experimental study with indoor fan wind and an ML K-nearest-neighbours algorithm to estimate airspeed. Crowe [13] demonstrated both of these models experimentally in the field, applying a similar LSTM model to a real drone with orientation and acceleration as inputs in hovering conditions. We studied this problem experimentally in [14], expanding the number of inputs to include the full drone state and control inputs by measuring the rotor speeds in flight. It was shown that accuracy improved when utilizing a “hybrid” approach, that utilized drag estimations from known equations of motion as an additional model input. However there was a decrease in performance between randomly selected and complete flight performance metrics, indicating a lack of generalizability.

In this paper, we improve upon the drawbacks of previous of previous work by considering the effects of data rotation

and reduction to improve generalizability. Data rotation enforces rotational invariance in the trained ML models, while data reduction eliminates data imbalance issues and decreases the number of training examples needed to sufficiently train a model.

This paper is organized as follows: first the previous work is reviewed in Section II, explaining the theory and experiment which are consistent with this work and explaining the drawbacks of previous methods. Section III explains how rotation and reduction are applied, and how the training, validation and testing data sets are selected. The models are trained, and performance compared in Section V. Finally in Sections VI and VII, the paper concludes with a discussion of significance and future work.

II. REVIEW OF PREVIOUS WORK

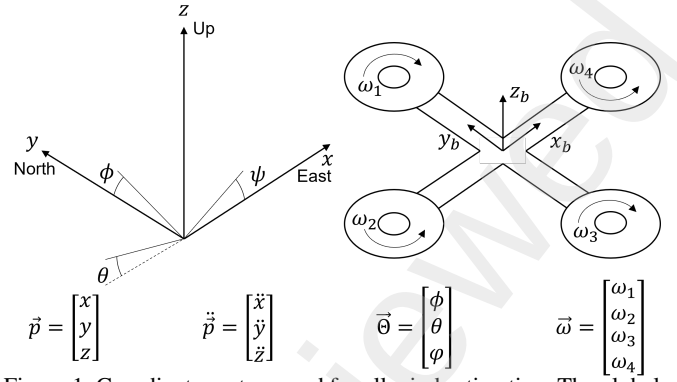
In [14], we provided a derivation of the theoretical relationship from the drone state to the wind. As this work is a direct improvement to that work, it is concisely summarized here. The main drawbacks of this previous work are addressed, which is the primary motivation for developing data rotation and reduction procedures.

A. Wind Estimation Problem

An understanding of the physical relationships between drone state and wind is useful in developing data augmentation and reduction procedures. The following relationship is derived in the coordinate frames established by Figure 1, showing the difference between the global and body reference frames. The relationship between drag and drone state is

$$m \begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ -mg \end{bmatrix} + R \begin{bmatrix} 0 \\ 0 \\ F \end{bmatrix} - \begin{bmatrix} D_x \\ D_y \\ D_z \end{bmatrix}, \quad (1)$$

where m [kg] is the mass of the drone, g [m/s²] is the acceleration due to gravity, and F [N] is the total thrust from the propellers which is a quadratic function of the rotor speeds [15], denoted by $[\omega_1, \omega_2, \omega_3, \omega_4]'$ [rad/s]. $\vec{D} = [D_x, D_y, D_z]'$ [N] is the drag force acting on the drone due to airspeed. R is the transformation matrix from the body frame to the global frame of reference and is a function of the orientation of the drone, which is omitted for brevity but described in [15]. The drone's position in global coordinates is represented by $\vec{p} = [x, y, z]'$ as shown in Figure 1, meaning that $\ddot{\vec{p}} = [\ddot{x}, \ddot{y}, \ddot{z}]'$ is the acceleration in global coordinates. It is important to note that this equation is derived in the global frame of reference, which is why the rotor force, F , which acts in the body frame is translated to the global frame by R . By rearranging (1), the disturbance drag can be estimated by full knowledge of the drone state, namely by measuring the total thrust (or rotor speeds), the acceleration, and the orientation of the drone. The drag is then related to the airspeed by a drag coefficient relationship. However, this requires knowledge of how the drag coefficient area product varies with drone orientation in three dimensions. This plant parameter is not trivial to obtain, which is the primary motivation for applying ML to this problem. Finally,



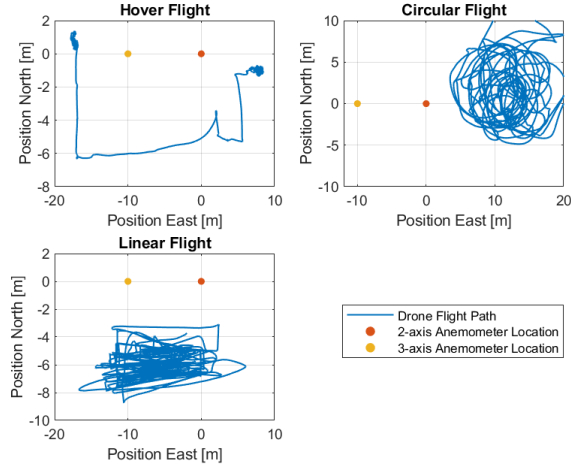


Figure 2: Typical flight paths used for data collection, relative to the 2-axis and 3-axis anemometers.

take $\hat{\vec{V}}_W = [\hat{V}_{WX} \hat{V}_{WY}]'$, which estimates wind in Cartesian coordinates. Cartesian coordinates are preferred to polar, to avoid the discontinuity in direction.

B. Experiment

The experimental platform is largely the same as used in [14], but is summarized here. The primary difference is the use of a second anemometer. To collect flight data, a modified quadrotor IndroRobotics Scout Mk IIIB was used with added sensors. The internal IMU was used to sample acceleration ($\ddot{\vec{p}}$) and orientation ($\ddot{\vec{\Theta}}$), utilizing the onboard dataflash logs to sample at rates of 25 Hz and 10 Hz respectively. An real-time-kinematic (RTK) ground-positioning-system (GPS) was added as an additional sensor, which provided more accurate ground speed ($\dot{\vec{p}}$) and position ($\vec{p}$) measurements. This utilized a GPS receiver ground station, which sent correction messages GPS receiver on the drone over radio. Custom motor speed sensors based on hall effect sensors on a ring printed-circuit-board (PCB) were used for each motor, to measure the rotor speeds ($\vec{\omega}$) and to compute the thrust (F).

Two ultrasonic anemometers, namely a Gill Instruments WindSonic 60 and WindMaster, are used to measure the true wind speed ($\vec{V}_W$); the first being a 2-axis anemometer and the second a 3-axis. Both instruments have a measurement resolution of 0.01 m/s, and an accuracy of $\pm 2\%$ @ 12 m/s. They use a time-of-flight sensing principle that eliminates temperature effects on the speed of sound. The 3-axis anemometer was placed approximately 10 m due west of the 2-axis anemometer. Two anemometers are used here, as they give an estimate of the spatial variation of the wind, which in turn acts as the limiting factor of the wind estimation accuracy. This is further discussed in Section V-C. The 2-axis anemometer was set to sample at its maximum rate of 4 Hz, and the 3-axis 10 Hz but was down sampled in post processing to match the 2-axis measurements.

Data was collected over 7 days of test flights, allowing for varying wind conditions and directions. A total of 18 test flights were conducted across all days, with varying

flight lengths of up to 10 minutes. In total, 151 minutes of test flight data was collected and used for model training, validation and testing. Different flight paths were collected and used for training and model validation, with common flight paths shown in Figure 2. An example picture of the drone in flight is shown in Figure 3. In each of these flights, the distance between the drone position and the 2-axis anemometer was kept comparable to the distance between the two anemometers. The goal with these flight paths is to keep the differences in wind from the anemometers location to the drones location low, whilst avoiding the anemometer measuring the downwash of the drone itself. This hopes to reduce the error introduced by the spatial variation of the wind field.

C. Previous Results

In [14], we trained a variety of ML models, the best of which was the LSTM using the hybrid physics-ML approach which achieved 0.34 m/s root-mean-square-error (RMSE) or 48% normalized-root-mean-square-error (NRMSE) on randomly selected training data and 0.55 m/s RMSE or 57.5% NRMSE on complete, unseen test flights. The NRMSE was normalized by dividing by the root-mean-square (RMS) of the true wind signal, which accounts for varying magnitude of signals. The primary drawback of this work is this decrease in performance when moving from randomly selected data to new complete flights, which is how the models would be used in reality. Two issues are identified that likely cause this drawback. One issue is the lack of rotational invariance considerations in approximating a rotationally symmetric set of equations, and the other is data imbalance issues in the original training data set. These issues are detailed in the following subsections.

1) Rotational Invariance

The equations of motion are inherently invariant to rotations of the global reference frame, but the ML models are not. That is, the equations of motion work in any global coordinate frame. Since we are concerned with horizontal wind estimation only, we are only interested in implementing rotational invariance about the global Z-axis, which is aligned with gravity. Intuitively this feature is understood by thinking about a drone moving at a given direction with respect to the wind direction, which is shown in Figure 4. Physically,



Figure 3: Picture of flying drone in hover near the 2-axis anemometer used.

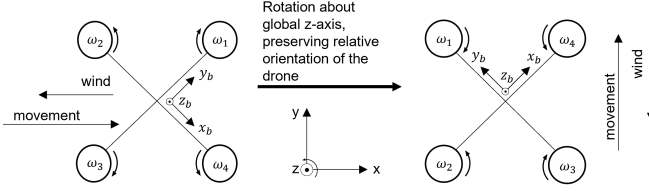


Figure 4: Illustration of rotational invariance property, which applies to the global reference frame but not the body reference frame. One rotor is colored to show that the drone is rotating to keep the relative direction preserved.

there is nothing different about this system when rotating by a given angle about the global Z-axis, which preserves the relative angle of the wind to the drone orientation. This effect does not however apply to the body frame of reference, which is fixed to the drone frame. We cannot expect to rotate the wind but not the drone and expect the system to behave the same, due to the different drag coefficient characteristics in different orientations. Considerations of rotational invariance are not included in previous work.

2) Data Imbalance

As demonstrated by (1), a compact way of representing the drone state which accounts for acceleration, orientation and rotor speeds is the drag estimation which is computed using the inverse equations of motion. We convert this global drag estimation to the body frame of reference using

$$\begin{bmatrix} D_{bx} \\ D_{by} \\ D_{bz} \end{bmatrix} = R^{-1} \begin{bmatrix} D_x \\ D_y \\ D_z \end{bmatrix}, \quad (5)$$

where $\vec{D}_b = [D_{bx}, D_{by}, D_{bz}]$ represents the drag estimate in the body frame. This is used to plot a two-dimensional histogram of the drone state (without true wind labels) in Figure 5. This shows an imbalance in the data set favouring low drag conditions, improperly weighting the training examples. The reason for this imbalance comes from practical flight planning. Naturally, the pilot will spend more time loitering and hovering for safety reasons than in dynamic flight phases.

III. DATA PROCESSING

To combat these two identified issues, we consider two data processing approaches (namely data augmentation by rotation and reduction) in order to select appropriate training, validation and testing data sets for the ML models. Subsection III-A describes the procedures used to augment the data, by considering invariance about rotations in the global Z-axis. Subsection III-B describes how this rotated data set is reduced to apply appropriate weighting and to reduce train time. Finally, subsection III-C summarizes these steps and explicitly states the selection of training, validation and testing data sets for each model.

A. Data Rotation

The previously described rotational invariance property is not inherent to ML models, and must either be learned

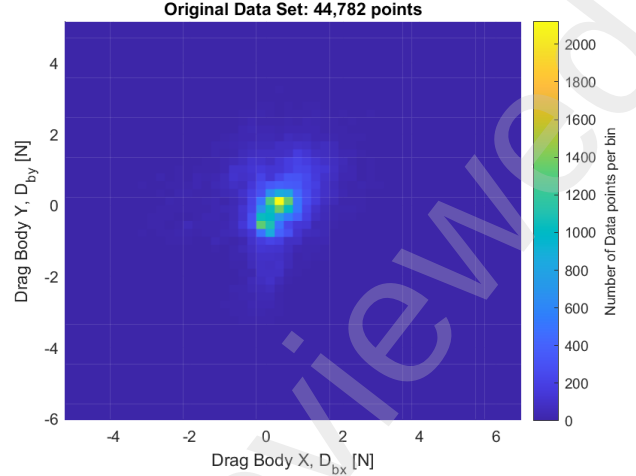


Figure 5: Histogram of representation of the drone state, from original data set which was used for training in [14].

through sufficient training examples or enforced in model implementation [16]. Here, we consider data rotation in order to teach the trained ML models rotational invariance about the global Z-axis. By doing so, we expect to increase the generalizability of these results as the estimations are no longer tied to the coordinate frame used in the particular training set. We rotate the data set a total of $k = 10$ times (including the original data set) over 2π , resulting in 0.62 rad or approximately 36° per rotation. Formally, this is implemented by the following 2-dimensional rotations:

$$\begin{aligned} i &= 2, 3, \dots, k, \\ \alpha^i &= \frac{2\pi}{n}(i-1), \end{aligned} \quad (6)$$

$$\begin{bmatrix} V_{WX}^i \\ V_{WY}^i \end{bmatrix} = \begin{bmatrix} \cos(\alpha^i) & -\sin(\alpha^i) \\ \sin(\alpha^i) & \cos(\alpha^i) \end{bmatrix} \begin{bmatrix} V_{WX} \\ V_{WY} \end{bmatrix}, \quad (7)$$

$$\psi^i = \psi + \alpha_i, \quad (8)$$

$$\begin{bmatrix} \dot{x}^i \\ \dot{y}^i \end{bmatrix} = \begin{bmatrix} \cos(\alpha^i) & -\sin(\alpha^i) \\ \sin(\alpha^i) & \cos(\alpha^i) \end{bmatrix} \begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix}, \quad (9)$$

$$\begin{bmatrix} \ddot{x}^i \\ \ddot{y}^i \end{bmatrix} = \begin{bmatrix} \cos(\alpha^i) & -\sin(\alpha^i) \\ \sin(\alpha^i) & \cos(\alpha^i) \end{bmatrix} \begin{bmatrix} \ddot{x} \\ \ddot{y} \end{bmatrix}, \quad (10)$$

where the superscripts i represent the i th rotation for each variable. All other variables are not modified as they will be the same for each rotated coordinate frame, namely the vertical components of velocity and acceleration, the pitch, the roll, and all rotor speeds. This is well described by Figure 6, which shows how the drag estimate varies when the global reference frame is rotated. In body coordinates, this representation does not change. The downside of this operation however is that the number of training examples has increased k fold, going from 44,782 to 447,820 which greatly increases the training time.

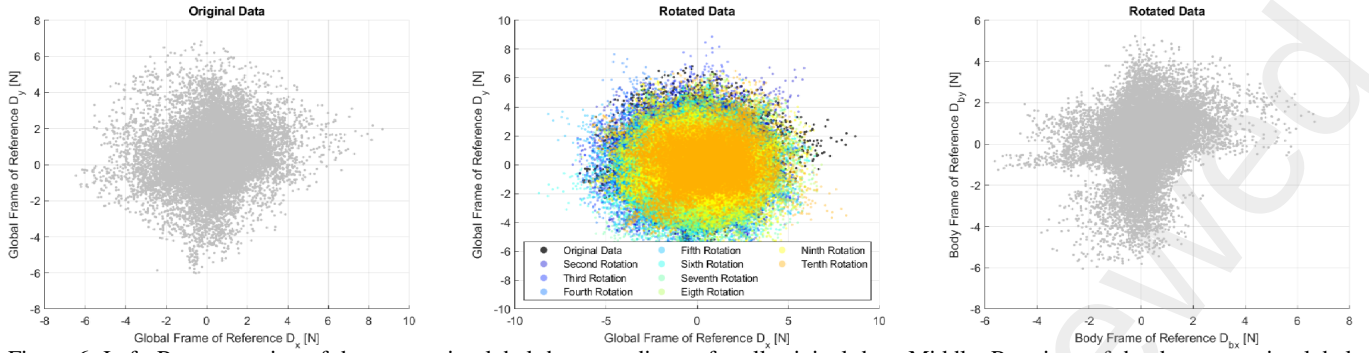


Figure 6: Left: Representation of drone state in global drag coordinates for all original data. Middle: Rotations of the drone state in global drag coordinates for 10 rotations over the full circle. Right: Representation of the drone state in body drag coordinates, which doesn't change with rotation of the global reference frame.

B. Data Reduction

The purpose of data reduction is to remove data imbalance issues highlighted by Section II-C2 and to reduce the total training time by reducing the number of training examples without significantly affecting performance. Data is reduced by enforcing an approximately even data point density in the body frame drag coordinates, by applying the grid based data selection method described by Jules [17]. This space was divided up into an evenly spaced grid, where only one data point was randomly selected to be kept for the reduced data set per cell. The rest of the points in each cell were discarded. Random selection preserves the non-linear relationship obeyed by each data point, which is not true of an ensemble method like averaging. This process is well described by Figure 7. The number of cells was selected to be 300, which gave approximately even data point density between the training and test sets, the selection of which is described in the next subsection. The reduced training data set is shown in Figure 8. While this method is effective for reducing the training time and applying more favourable weighting, the downside is that the training procedure is more sensitive to the particular noise and errors of this reduced set. Jules describes multiple methods for dealing with this imbalance problem, the most complex of which is by adding a weight vector computed by euclidean distance metrics. This was not pursued as it would not reduce the rotated data set, requiring extensive training time. Additionally MATLAB was used for model training, which doesn't have built-in functionality for specifying a weighted loss function in NN training.

C. Training, Validation and Testing Data

Data rotation and reduction are both applied in order to produce appropriate training and validation data sets. Starting from the original data set, which comes from all test flights performed, this data is rotated and appended corresponding to a tenfold increase in length. The testing data set is selected as three complete test flights from the original unrotated data, each of which correspond to a representative flight condition. Namely, one test flight with hover in low wind speed conditions, another flight in hover with high wind speed, and another flight with high ground speed and high

wind speed were used as the total testing set. Then from the remainder of the total data, data reduction is performed to produce a training data set. From the remainder of this operation, data reduction is performed again to produce a validation data set. By doing so, we produce a training data set with short enough length to provide fast training time, and without heavily imbalanced training examples.

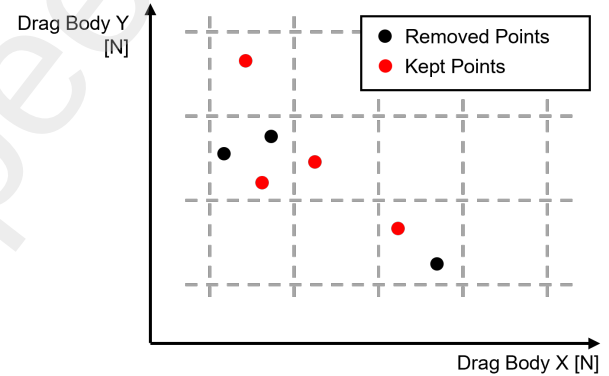


Figure 7: Description of Grid based data reduction technique used for decreasing the size of the data set and applying weighting. Similar to Figure in [17].

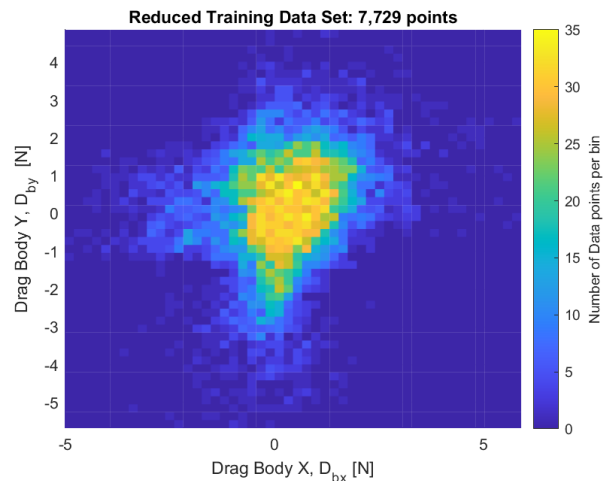


Figure 8: Histogram of data set after applying grid based data selection. Note the significant variation in the color bar relative to Figure 5.

The validation data set represents a similar data set to the training set in terms of state distribution and size, but is still unseen by the training procedure. It also comes from the total rotated data, meaning that it is a measure of how the model will perform on generalized data in any coordinate frame. Random selection is typically used when developing ML models, to avoid bias in performance assessment. However, in reality one would apply such a model to new, complete flight data which is what the testing data set represents. By using complete, non-random data sets for test we measure how well the model generalizes to new flights. Random selection is used in selection of the validation set, allowing for comparison between these metrics. This entire process is well summarized by Figure 9.

In order to demonstrate the effectiveness of these rotation and reduction techniques, multiple variations of these models are developed without these features and compared. In the case of models trained using reduction but not rotation, the training set is selected by performing data reduction on the remainder of the original unrotated data set alone. The validation data set is still selected from the rotated data, to show that this method does not generalize well to new coordinate frames. For models trained without rotation or reduction, the training set is taken as the total remainder of the original unrotated data set, with the same validation set. In all cases the testing and validation data sets are the same, to provide an effective comparison.

IV. MACHINE LEARNING MODEL DEVELOPMENT

Given the determined training, validation and testing data sets, this section describes the process used to train all ML models and their variations. Two main models are considered here: an LSTM model and a GRU model. LSTM's have been shown to be effective in this application by multiple sources [13, 11, 14], as they are optimized for time series analysis. A GRU model is considered as they are a simplification from the typical LSTM structure but still perform well on time series problems. While an LSTM unit is composed

of three gates, namely and input, output and forget gate, the GRU combines the forget and input gates into a single update gate. GRU's are typically more computationally efficient than LSTMs, but with less complexity [18]. For both models, three variations are developed: one with data rotation and reduction applied, one without reduction but not rotation, and one with neither rotation nor reduction.

A. LSTM Model

LSTM models are an improvement upon general recurrent-neural-networks, as they are able to hold information from previous time steps in an internal cell state for an arbitrarily number of time steps. They are a generalized approach to time series machine learning problems. To develop the LSTM NN, the same structure and training procedure was followed as in previous work [14], but is summarized here. Two hidden layers are used with 90 and 48 units each. Each layer uses L_2 regularization with a strength of 1×10^{-4} . No dropout layers were used as it was found that overfitting typically did not occur, which is seen by the training history plot in Figure 10. The Adam optimizer was used with an initial learning rate of 1×10^{-3} which dropped by 80% every 10 epochs until a maximum of 50 epochs. After this training, the training history was checked to see that overfitting did not occur. A mini-batch size of 30 was used. All model training was done using MATLAB, and was executed on remote clusters using Compute Canada resources.

B. GRU Model

A GRU NN is a simplification of an LSTM model, generally reducing the complexity of the model with faster training times and improved generalizability on less data. For consistency, the same structure and training procedure was used as the LSTM, that is two hidden layers with 90

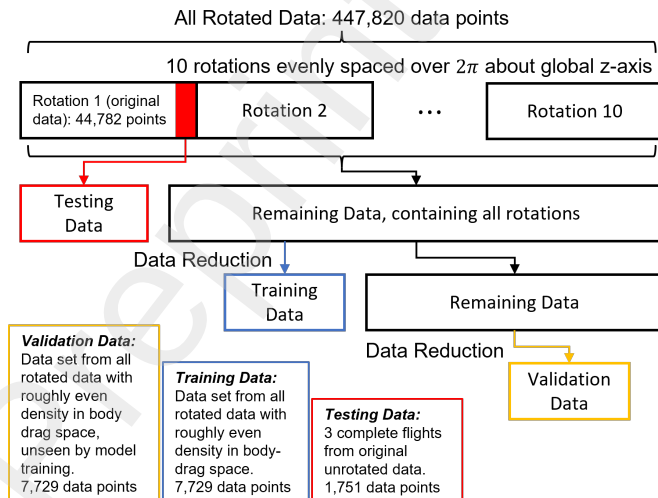


Figure 9: Summary of how training, validation and testing data sets were selected and reduced from the original rotated data set.

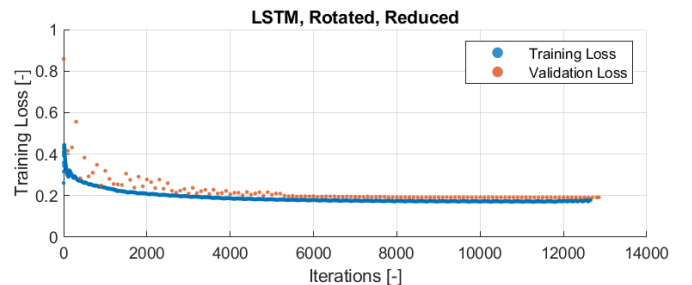


Figure 10: Training history of the LSTM NN with rotation and reduction applied.

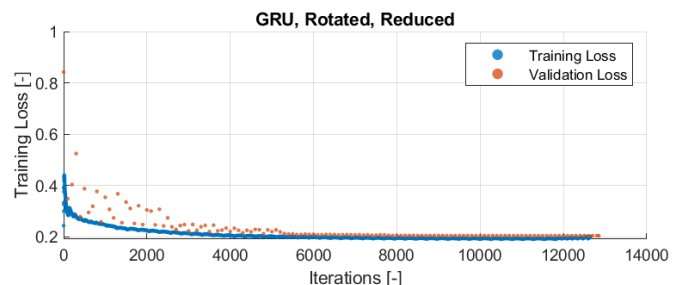


Figure 11: Training history of the GRU NN with rotation and reduction applied.

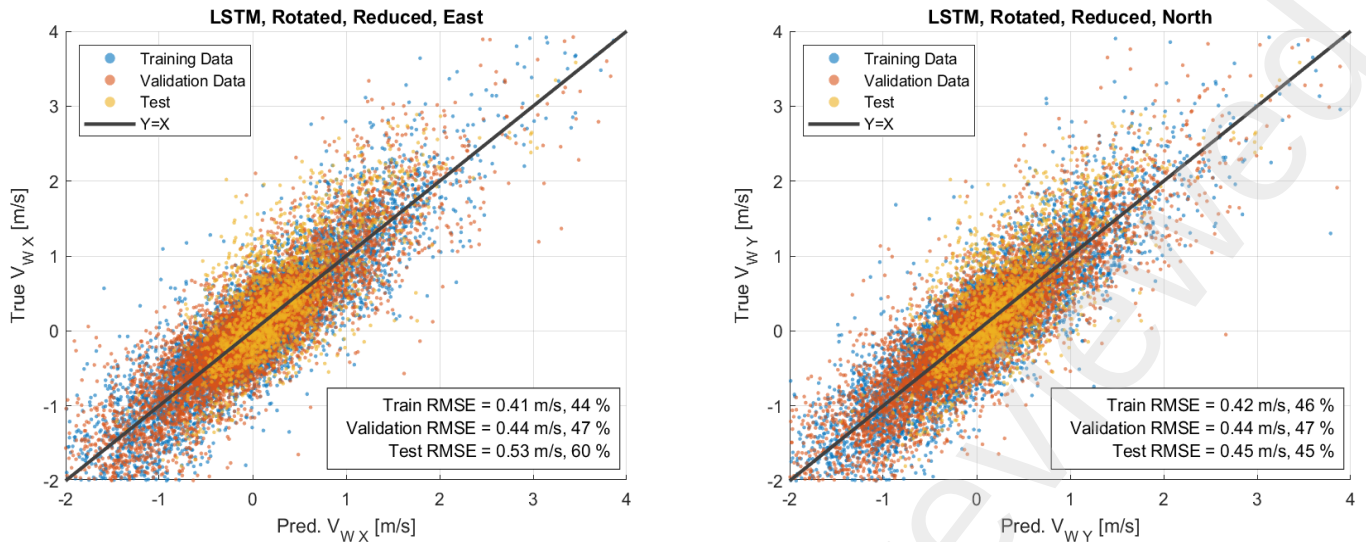


Figure 12: Scatterplot showing summary of performance of LSTM model with rotation and reduction applied.

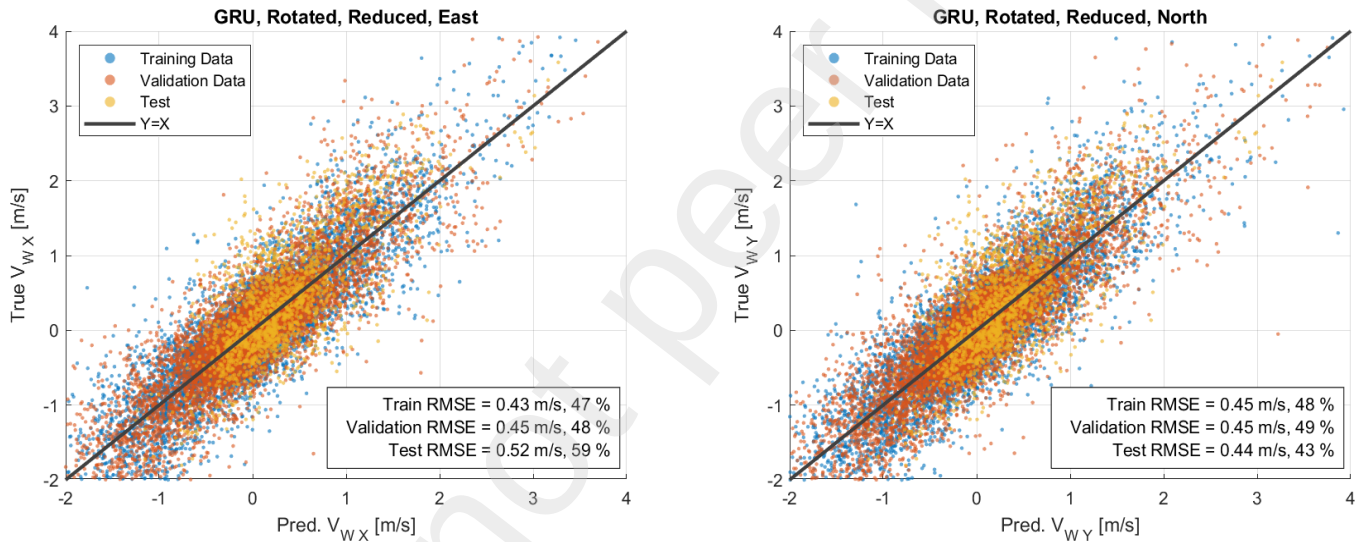


Figure 13: Scatterplot showing summary of performance of GRU model with rotation and reduction applied.

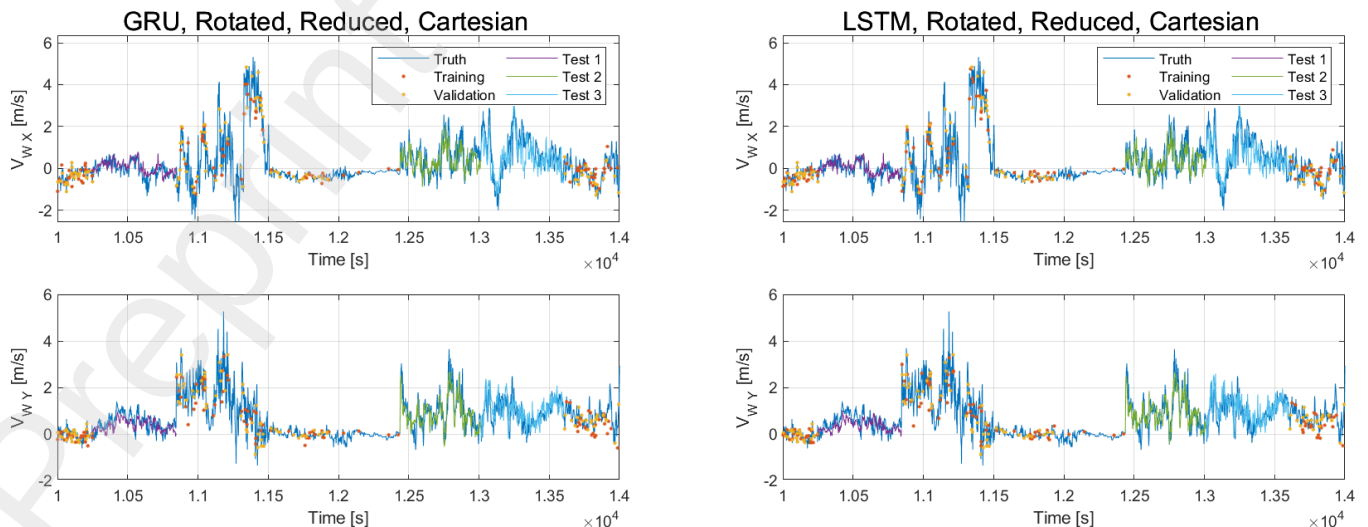


Figure 14: Left: Time series of 3 full test flights in testing data set for GRU model with rotation and reduction applied. Right: Same time series plot but for LSTM model.

Model	X Validation RMSE	X Test 1 / 2 / 3	Y Validation RMSE	North Test 1 / 2 / 3
LSTM, original	0.77 m/s (84%)	0.27 / 0.61 / 0.72 m/s (78% / 71% / 59%)	0.88 m/s (93%)	0.27 / 0.56 / 0.64 m/s (46% / 45% / 58%)
LSTM, reduced	0.75 m/s (83%)	0.30 / 0.69 / 0.83 m/s (84% / 81% / 68%)	0.83 m/s (87%)	0.29 / 0.61 / 0.71 m/s (48% / 49% / 64%)
LSTM, rotated and reduced	0.44 m/s (47%)	0.29 / 0.56 / 0.66 m/s (83% / 65% / 55%)	0.44 m/s (47%)	0.31 / 0.51 / 0.51 m/s (51% / 41% / 47%)
GRU, original	0.77 m/s (84%)	0.27 / 0.61 / 0.72 m/s (78% / 71% / 59%)	0.88 m/s (93%)	0.27 / 0.56 / 0.64 m/s (46% / 45% / 58%)
GRU, reduced	0.75 m/s (83%)	0.27 / 0.65 / 0.77 m/s (78% / 76% / 63%)	0.82 m/s (86%)	0.29 / 0.56 / 0.64 m/s (49% / 45% / 59%)
GRU, rotated and reduced	0.45 m/s (48%)	0.30 / 0.54 / 0.65 m/s (86% / 64% / 53%)	0.45 m/s (49%)	0.30 / 0.51 / 0.48 m/s (49% / 41% / 44%)

Table I: Comparison of validation and test performance metrics for the LSTM and GRU models, and their variations.

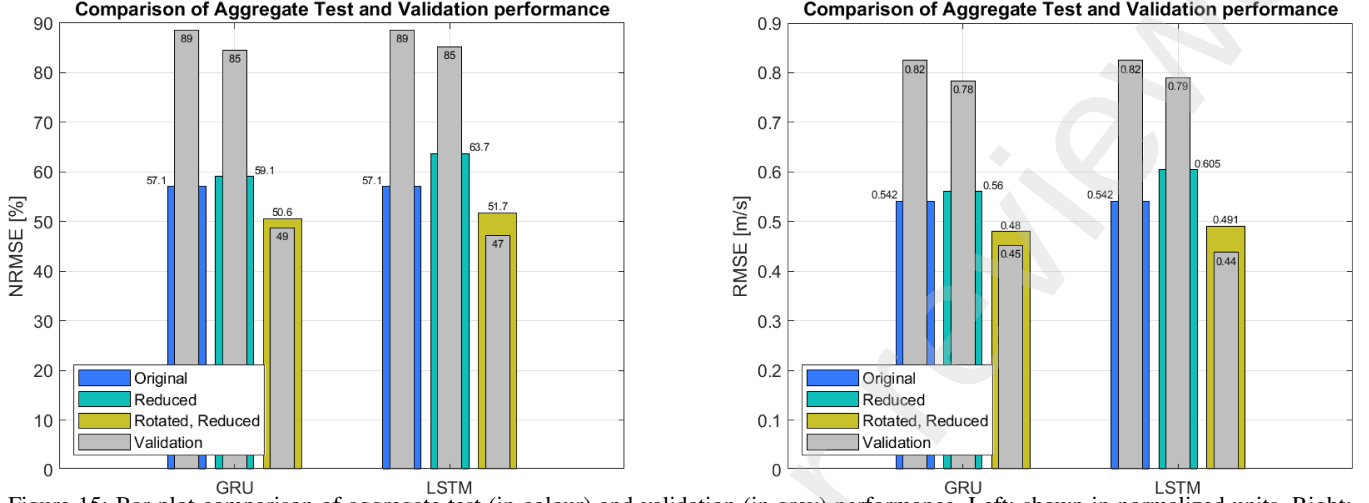


Figure 15: Bar plot comparison of aggregate test (in colour) and validation (in grey) performance. Left: shown in normalized units. Right: shown in absolute units.

and 48 units each with 1×10^{-4} L_2 regularization. The same training optimizer, learning rate schedule and batch size are used. The training history for the GRU is shown in Figure 11 which again shows monotonically decreasing loss which never increases. Both models achieve similar validation performance, meaning that we expect both models to perform similarly.

V. PERFORMANCE EVALUATION

Performance for each trained model is determined by accuracy, as measured by the root-mean-square-error (RMSE), on the validation and test data sets as well as the total training time. RMSE is used as the primary performance metric, as it provides an assessment of variance and bias errors and is in absolute units of m/s which is intuitive. Since this error is proportional to the size or variation of a signal, it is not useful for comparing different data sets. For these comparisons, the normalized-root-mean-square-error (NRMSE) is used, which is calculated as the RMSE divided by the root-mean-square (RMS) of the true signal.

A. Model Accuracy

Figures 12 and 13 show a scatterplot comparison between the estimated wind values and true wind values in each principle coordinate of the global reference frame. Performance is quantified in the training error, the validation error, and test errors all in absolute and normalized units. For clarity, only plots are shown for the LSTM and GRU rotated and reduced variations, but all performance metrics are summarized in Table I. A time series comparison is shown for both the LSTM and GRU in Figure 14, indicating the segmentation between the test data sets and the grid reduction

selection of the training and validation sets. Since there are multiple performance metrics to track, Figure 15 shows the aggregate test and validation performance metrics for each model providing a simpler comparison. This data is shown with both absolute and normalized units.

There is a very clear benefit to providing data rotation, as it effectively improves the estimation in generalized global coordinates which is seen by the large difference in validation performance. The test performance metric, which only consists of data from the original data set, improves too but by not as much. This indicates that rotating and reducing the data set improves the generalization of the results into new, complete flights. The LSTM reaches the best validation performance of 0.44 m/s RMSE or 47% NRMSE, while the GRU achieves the best test performance of 0.48 m/s RMSE or 50.6% NRMSE. The validation performance reached here is worse than that reached by [14], but the test performance is better for the same data. The similarity between the test and validation performance metrics likely indicates that this is much closer to realistic performance given sensor biases and day-to-day variations.

B. Model Training Time

In addition to comparing the accuracy performance benefits of rotating and reducing the data set, we consider the benefits in the training time which is primarily due to the decreased number of data points used for training. This comparison is shown in Table II. The most significant difference in training time is between the variations with data reduction, and the variation without with an increase in training time of at least six times relative to the fastest trained model. It should be noted that training models on the entire

Total Training Time	LSTM	GRU
Original	1733 s	1775 s
Reduced	269 s	268 s
Rotated, Reduced	335 s	324 s

Table II: Comparison of Training time required for each model.

rotated data set without reduction applied took an excessive amount of time, over 8 hours, which is why these models were not developed for comparison.

C. Limits of Performance

As this is an experimental study, the sources of error limit the achievable accuracy of the developed models. In our analysis we have assumed a spatially homogeneous wind field which only varies in time to obtain true wind speed labels, the validity of which we investigate here. As previously described in Section II-B, two ultrasonic anemometers placed at different locations when flying the drone are used to assess the spatial variation of the wind. This method for measuring the spatial variation is much more effective than previously considered convection models [14].

Figure 16 compares the measurements by the 3-axis anemometer, the 2-axis anemometer and the predictions by the GRU model with data rotation and reduction applied. This is a representative time series in order to show fine details, but in reality this time series is longer. This comparison is made on the complete original data set with both anemometers signals available, including training examples which is generally bad practice. This is not expected to be an issue, as the data reduction process leaves an excess of unseen data and training data only accounts for 1.7% of this data set. We expect the variance between each signal to be comparable, which is generally true except for brief deviations. For example, there is a temporary deviation around 2800 seconds in global Y wind estimation. It is unknown whether these deviations are a failure of the model, a gust of wind experienced by the drone but not the anemometers, or simply a particular sensors inaccuracy issue. To quantify the comparison between the anemometers and the drone prediction, we compute the RMSE of differences between the 2-axis and 3-axis anemometer and compare this to the errors within the model predictions. This is shown in Figure 17, which also considers how this RMSE decreases with increasing averaging filter time. This comparison is useful for showing what level of accuracy is attainable when less-frequent measurement is required. When applying a 100 second averaging filter, the models reach estimation accuracies of less than 0.2 m/s RMSE. For comparison, Crowe's results are included as well, however it is important to note that we do not have the time series data for this performance available meaning that we cannot normalize these results for comparison.

VI. DISCUSSION

These results show the possible accuracy performance achievable by multirotor drones estimating wind by use of a ML based disturbance observer. In this paper we have taken as a given that the hybrid set of drone state inputs yields good results, which is thoroughly discussed in [14].

The primary investigation is the effects of applying data reduction and rotation, which decrease the required training data, decrease the training time, and improve model performance over generalized data sets. Validation and performance metrics were selected that showed how the models performed on both randomly selected data and complete test flights. While random selection is very common in assessing ML models to avoid bias, there is likely variation in sensor performance between flights and including unseen complete flights assesses how our ML models generalize to this.

While a large number of data points were collected and then reduced to a reasonable number for training, this procedure is not required when replicating or applying this work. We can look at the method for reducing the data set as a framework for what types of test flights are important for training these ML models. Generally, it is desirable to achieve an approximately uniform density of data points in body frame drag coordinates which can be obtained by flying in high wind conditions or flying aggressively in low-wind

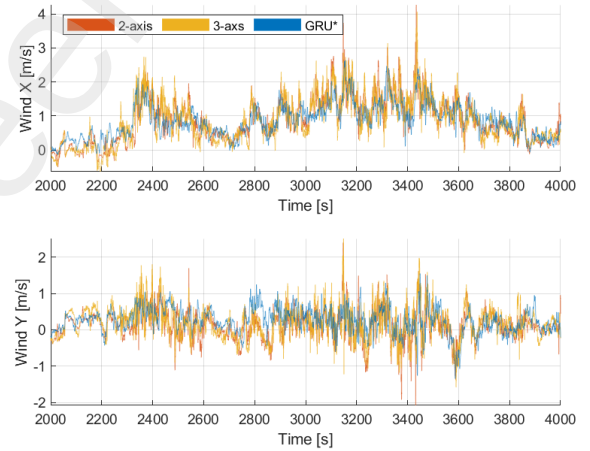


Figure 16: Comparison of time series of 3-axis anemometer reading, 2-axis anemometer reading, and GRU model estimations.

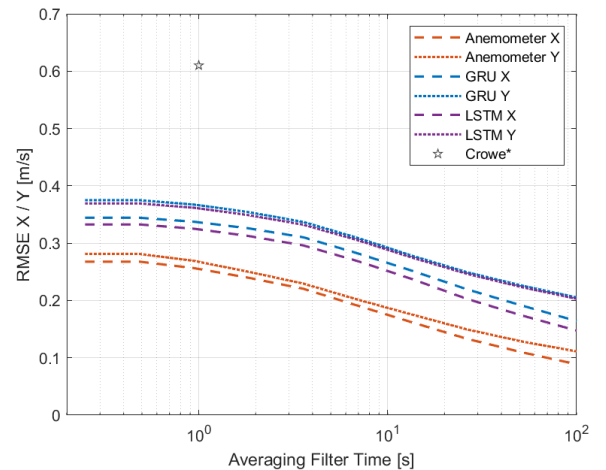


Figure 17: Comparison of how the spatial variation of the wind, and the model estimation accuracy decrease with increasing averaging time, in absolute units. *Crowe [13].

conditions. Inevitably due to take-off, landing, and loitering, there may be an excess of data points corresponding to low-drag conditions which can be addressed by the same reduction technique, or time-based data segmentation and removal. During flight, it is important to fly the drone in varying orientations with respect to the airspeed direction in order to fully explore the body-drag space. Intuitively this is understood by the fact that the models learn the drag characteristics of the drone in an arbitrary orientation, which is achieved by including training examples of the drone experience ranges of airspeeds in all orientations.

While Figures 16 and 17 indicate that the spatial variation of the wind is a significant factor in determining the limits of performance given our experimental method, there is still a gap of about 20% between the anemometer variance and model accuracy. This could be caused by spatial variation not captured by the two anemometers, or model / sensor input error. An important factor is that the same configuration of anemometers was used for varying wind conditions, where the primary wind direction is changing. We expect a higher spatial correlation of wind when measured parallel to the primary direction of wind rather than perpendicular [19]. Other sources of error include variation in sensor performance from day to day, which the ML models assume is uniform. For example, the GPS accuracy depends on how long the ground station has been stationary for. Therefore, if multiple test flights are conducted consecutively the last test flight will have higher GPS accuracy than the first.

VII. CONCLUSIONS AND FUTURE WORK

In summary, we have presented a ML based disturbance observer for multirotor drones which can be used to implicitly measure the wind field, a tool useful in environmental monitoring, wind farm site surveying and other applications. The developed disturbance observer builds upon previous works by improving the accuracy over generalized flight conditions, applying data rotation and reduction techniques. Data rotation allows the ML models to learn rotational invariance, by including training examples in arbitrary global coordinates. Data reduction provides appropriate weighting to training examples, by removing data point imbalances which originally favoured low-drag conditions. This also offers the benefit of reducing the training time significantly relative to training on unreduced data. The trained GRU model reached 0.48 m/s RMSE on unseen testing data, which was comprised of complete test flights, which properly assesses how this model will perform in realistic generalized conditions.

Given the current experimental method, performance is limited by the spatial variation of wind. It is recommended that future work be focused on improving the experiment rather than changing or tuning the models used, as this will likely yield the most significant improvements to performance. Either observed laminar flow conditions in the field or enforced in a wind tunnel can be used to decrease the spatial variation, although use of large wind tunnels with room for dynamic flight are much less commonly available to researchers. A more expensive LIDAR anemometer that

can measure the wind field over spatial dimensions could be used in the field to improve the method. Additionally, this work represents a demonstration of wind estimation by multirotor drone that uses post processing exclusively. This is suitable for some applications, but not when information is to be used for real-time decision making. It would be highly valuable to demonstrate a real-time disturbance observer, making autonomous decisions about localizing unknown gas sources or modifying planned flight trajectories based on the estimated wind.

ACKNOWLEDGEMENTS

Acknowledgements omitted in anonymized manuscript.

REFERENCES

- [1] Yibo Sun et al. "A UAV-Based Eddy Covariance System for Measurement of Mass and Energy Exchange of the Ecosystem: Preliminary Results". In: *Sensors* 21.2 (2021). ISSN: 1424-8220. DOI: 10.3390/s21020403. URL: <https://www.mdpi.com/1424-8220/21/2/403>.
- [2] Derek Hollenbeck, Demetrius Zulevic, and Yangquan Chen. "Advanced Leak Detection and Quantification of Methane Emissions Using sUAS". In: *Drones* 5.4 (2021). ISSN: 2504-446X. DOI: 10.3390/drones5040117. URL: <https://www.mdpi.com/2504-446X/5/4/117>.
- [3] Adil Shah et al. "A Near-Field Gaussian Plume Inversion Flux Quantification Method, Applied to Unmanned Aerial Vehicle Sampling". In: *Atmosphere* 10.7 (2019). ISSN: 2073-4433. DOI: 10.3390/atmos10070396. URL: <https://www.mdpi.com/2073-4433/10/7/396>.
- [4] Jacob Shaw et al. "Methods for quantifying methane emissions using unmanned aerial vehicles: a review". In: *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences* 379 (2021). URL: <https://doi.org/10.1098/rsta.2020.0450>.
- [5] Matthew Marino et al. "An Evaluation of Multi-Rotor Unmanned Aircraft as Flying Wind Sensors". In: *International Journal of Micro Air Vehicles* 7.3 (2015), pp. 285–299. DOI: 10.1260/1756-8293.7.3.285. eprint: <https://doi.org/10.1260/1756-8293.7.3.285>. URL: <https://doi.org/10.1260/1756-8293.7.3.285>.
- [6] Norman Wildmann, Sarah Bernard, and Jens Bange. "Measuring the local wind field at an escarpment using small remotely-piloted aircraft". In: *Renewable Energy* 103 (Nov. 2016). DOI: 10.1016/j.renene.2016.10.073.
- [7] Vahram Stepanyan and Kalmanje S. Krishnakumar. "Estimation, Navigation and Control of Multi-Rotor Drones in an Urban Wind Field". In: *AIAA Information Systems-AIAA Infotech @ Aerospace*. DOI: 10.2514/6.2017-0670. eprint: <https://arc.aiaa.org/doi/pdf/10.2514/6.2017-0670>. URL: <https://arc.aiaa.org/doi/abs/10.2514/6.2017-0670>.

- [8] Jack Langelaan, Nicholas Alley, and James Neidhoefer. "Wind Field Estimation for Small Unmanned Aerial Vehicles". In: *Journal of Guidance, Control, and Dynamics* 34 (July 2011), pp. 1016–1030. DOI: 10.2514/1.52532.
- [9] W. Thielicke et al. "Towards accurate and practical drone-based wind measurements with an ultrasonic anemometer". In: *Atmospheric Measurement Techniques* 14.2 (2021), pp. 1303–1318. DOI: 10.5194/amt-14-1303-2021. URL: <https://amt.copernicus.org/articles/14/1303/2021/>.
- [10] Patrick Neumann and Matthias Bartholmai. "Real-time wind estimation on a micro unmanned aerial vehicle using its inertial measurement unit". In: *Sensors and Actuators A: Physical* 235 (Nov. 2015), pp. 300–310. DOI: 10.1016/j.sna.2015.09.036.
- [11] Sam Allison, He Bai, and Balaji Jayaraman. "Wind estimation using quadcopter motion: A machine learning approach". In: *Aerospace Science and Technology* 98 (2020), p. 105699. ISSN: 1270-9638. DOI: <https://doi.org/10.1016/j.ast.2020.105699>. URL: <https://www.sciencedirect.com/science/article/pii/S1270963819324034>.
- [12] Liyang Wang, Gaurav Misra, and Xiaoli Bai. "A K Nearest Neighborhood-Based Wind Estimation for Rotary-Wing VTOL UAVs". In: *Drones* 3.2 (2019). ISSN: 2504-446X. DOI: 10.3390/drones3020031. URL: <https://www.mdpi.com/2504-446X/3/2/31>.
- [13] David Crowe et al. "Two Supervised Machine Learning Approaches for Wind Velocity Estimation Using Multi-Rotor Copter Attitude Measurements". In: *Sensors* 20.19 (2020). ISSN: 1424-8220. DOI: 10.3390/s20195638. URL: <https://www.mdpi.com/1424-8220/20/19/5638>.
- [14] Steven Zimmerman et al. "Wind Estimation by Multirotor Dynamic State Measurement and Machine Learning Models". In: *Measurement, under review* (2022).
- [15] Teppo Luukkonen. "Modelling and control of quadcopter". In: *Independent research project in applied mathematics, Espoo* 22 (2011), p. 22.
- [16] Julia Ling, Reese Jones, and Jeremy Templeton. "Machine learning strategies for systems with invariance properties". In: *Journal of Computational Physics* 318 (2016), pp. 22–35. ISSN: 0021-9991. DOI: <https://doi.org/10.1016/j.jcp.2016.05.003>. URL: <https://www.sciencedirect.com/science/article/pii/S0021999116301309>.
- [17] Jules Matz et al. "Parameter identification for nonlinear models from a state-space approach". In: *IFAC-PapersOnLine* 53 (Jan. 2020), pp. 13910–13915. DOI: 10.1016/j.ifacol.2020.12.905.
- [18] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. <http://www.deeplearningbook.org>. MIT Press, 2016.
- [19] Lars Morten Bardal and Lars Sætran. "Spatial correlation of atmospheric wind at scales relevant for large scale wind turbines". In: *Journal of Physics: Conference Series* 753 (Sept. 2016), p. 032033. DOI: 10.1088/1742-6596/753/3/032033.